

TP4 — Evidencias · Reportes analíticos asistidos por IA (Semana 4)

Parte 1 — Consulta 1 (facturación por categoría y mes, 4 tablas) · ANTES

SQL del ejercicio 1: detalle_pedido ⋈ pedido ⋈ producto ⋈ categoria

```
-- 1
EXPLAIN ANALYZE
SELECT to_char(date_trunc('month', p.fecha), 'YYYY-MM') AS mes,
       c.nombre AS categoria, sum(dp.subtotal) AS facturado
FROM detalle_pedido dp
JOIN pedido    p  ON p.id = dp.pedido_id
JOIN producto  pr ON pr.id = dp.producto_id
JOIN categoria c  ON c.id = pr.categoria_id
GROUP BY mes, c.nombre
ORDER BY mes, facturado DESC;
```

Tiempo real: 4387.557 ms

```
Incremental Sort (cost=114130.92..146885.70 rows=600000 width=72) (actual time=4141.474..4359.437 rows=60.0)
  → GroupAggregate (Group Key: mes, c.nombre) rows=600000 → 60
    → Sort (external merge, Disk: 20160kB) rows=600000
      → HASH JOIN (Hash Cond: pr.categoria_id = c.id) [algoritmo: HASH JOIN]
        Hash (rows=4) → Seq Scan categoria
      → HASH JOIN (Hash Cond: dp.producto_id = pr.id) [algoritmo: HASH JOIN]
        Hash (rows=50000) → Seq Scan producto
      → HASH JOIN (Hash Cond: dp.pedido_id = p.id) [algoritmo: HASH JOIN]
        Seq Scan detalle_pedido (rows 600000)
        Hash (rows=200000) → Seq Scan pedido
Execution Time: 4387.557 ms
```

Parte 1 — Consulta 1 · DESPUÉS (reescritura, ejercicio 3)

Tiempo real: 2052.146 ms · Mejora: 2,14x

```
-- 3
EXPLAIN ANALYZE
SELECT v.mes, c.nombre AS categoria, v.facturado
FROM ( SELECT to_char(date_trunc('month', p.fecha), 'YYYY-MM') AS mes,
              pr.categoria_id, sum(dp.subtotal) AS facturado
      FROM detalle_pedido dp
      JOIN pedido    p  ON p.id = dp.pedido_id
      JOIN producto  pr ON pr.id = dp.producto_id
      GROUP BY mes, pr.categoria_id ) v
JOIN categoria c ON c.id = v.categoria_id
ORDER BY v.mes, v.facturado DESC;
```

```
Sort (quicksort 27kB) rows=60 ← ya NO hay sort a disco de 20 MB
  → Hash Join (Hash Cond: pr.categoria_id = c.id) [algoritmo: HASH JOIN]
```

```
HashAggregate (Group Key: mes, pr.categoria_id) rows 600000 → 60
  → Hash Join (dp.producto_id = pr.id) [algoritmo: HASH JOIN]
    Hash Join (dp.pedido_id = p.id) [algoritmo: HASH JOIN]
      Seq Scan detalle_pedido
      Hash (rows=200000) → Seq Scan pedido
      Hash (rows=50000) → Seq Scan producto
      Hash (rows=4) → Seq Scan categoria
Execution Time: 2052.146 ms
```

Índice de cobertura aplicado (ejercicio 4)

```
-- 4
CREATE INDEX idx_detalle_cover_pedido_prod
ON detalle_pedido (pedido_id, producto_id) INCLUDE (subtotal);
```

Parte 1 — Consulta 2 (ranking clientes por gasto, 3 tablas)

ANTES (ejercicio 2)

Tiempo real: 753.714 ms

```
Limit (top-N heapsort, rows=20)
Sort (sum(dp.subtotal) DESC) rows 20000
Finalize HashAggregate (Group Key: cl.id, Batches: 5, Disk Usage)
Gather (Workers Planned: 2, Workers Launched: 2)
  Partial HashAggregate
    HASH JOIN (Hash Cond: p.cliente_id = cl.id) [algoritmo: HASH JOIN]
      PARALLEL HASH JOIN (Hash Cond: dp.pedido_id = p.id) [algoritmo: PARALLEL HASH JOIN]
        Parallel Seq Scan detalle_pedido
        Hash (rows=117647) → Parallel Seq Scan pedido
        Hash (rows=20000) → Seq Scan cliente
Execution Time: 753.714 ms
```

DESPUÉS (ejercicio 5, tras índice propuesto y descartado)

Sin mejora: 751.275 ms · el índice en pedido(cliente_id,fecha) no cambió el plan → propuesta descartada (registrada en bitácora)

Execution Time: 751.275 ms
(mismo plan: Parallel Hash Join + Hash Join; el cuello de botella es el spill del HashAggregate)

Parte 2 — Lectura crítica del plan de join

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
"El plan usa Nested Loop para unir detalle_pedido con pedido, buscando por índice en cada fila"	No	El plan real usa Hash Join en los tres nodos; no hay Nested Loop ni uso de índice en esas uniones.
"La tabla externa es detalle_pedido y por cada fila se busca en pedido por índice"	No	En Hash Join no hay externa/interna como en Nested Loop; el build es un Hash sobre pedido (rows=200000) y el

		probe es detalle_pedido.
"El tercer join contra categoria es el más caro y su costo 38088 es el tiempo total"	No	El build contra categoria es un Hash de 4 filas ; 38088 son unidades de costo estimado , no ms. Tiempo total real: 4387 ms .
"La agregación ordena 600.000 filas por mes y categoría"	Sí	El plan muestra Sort ... external merge Disk: 20160kB y GroupAggregate con ese Group Key.

3 imprecisiones reales detectadas sobre planes con varios JOIN.

Parte 3 — Ranking con función de ventana (spec a)

Especificación precisa

"Para cada cliente vigente con al menos un pedido: nombre completo, total gastado (suma de detalle_pedido.subtotal) y puesto en ranking de mayor a menor gasto, sin colapsar filas. Empates comparten el mismo puesto. No usés SELECT *."

Versión 1 — RANK() (ventana)

```
-- 3a
WITH g AS (
  SELECT p.cliente_id, sum(dp.subtotal) AS total
  FROM pedido p JOIN detalle_pedido dp ON dp.pedido_id = p.id
  GROUP BY p.cliente_id
)
SELECT cl.id, cl.nombre, cl.apellido, g.total,
       RANK() OVER (ORDER BY g.total DESC) AS puesto
FROM g JOIN cliente cl ON cl.id = g.cliente_id
ORDER BY puesto, cl.id;
```

Versión 2 — subconsulta correlacionada replicando RANK

```
-- 3b
WITH g AS ( ...mismo CTE... )
SELECT cl.id, cl.nombre, cl.apellido, g.total,
       (SELECT count(*) + 1 FROM g h WHERE h.total > g.total)::bigint AS puesto
FROM g JOIN cliente cl ON cl.id = g.cliente_id
ORDER BY (SELECT count(*) + 1 FROM g h WHERE h.total > g.total), cl.id;
```

Verificación de equivalencia:

```
-- 3c / 3d
(v1) EXCEPT (v2); → 0 filas
(v2) EXCEPT (v1); → 0 filas
```

EQUIVALENTES — 0 filas en ambas direcciones

Parte 3 — Subconsulta correlacionada (spec b)

Especificación precisa

"Para cada producto: nombre, precio y la diferencia entre su precio y el precio promedio de los productos de su misma categoría (subconsulta correlacionada).
Incluí todos los productos. No usés SELECT *."

Versión 1 — subconsulta correlacionada

```
-- 3e
SELECT p.id, p.nombre AS producto, p.precio,
       round(p.precio - (SELECT avg(p2.precio) FROM producto p2
                        WHERE p2.categoria_id = p.categoria_id), 2) AS dif_vs_categoria
FROM producto p; -- (LIMIT 50 solo para el preview)
```

Versión 2 — join + agregación (estructura distinta)

```
-- 3f
SELECT p.id, p.nombre AS producto, p.precio,
       round(p.precio - c.promedio, 2) AS dif_vs_categoria
FROM producto p
JOIN (SELECT categoria_id, avg(precio) AS promedio
     FROM producto GROUP BY categoria_id) c
ON c.categoria_id = p.categoria_id; -- (LIMIT 50 solo para el preview)
```

Verificación de equivalencia:

```
-- 3g / 3h
(v1) EXCEPT (v2); → 0 filas
(v2) EXCEPT (v1); → 0 filas
```

EQUIVALENTES — 0 filas en ambas direcciones

Parte 4 — Competencia de optimización

Equipo	Estrategia aplicada	Antes (ms)	Después (ms)	Mejora (x)
Nuestro equipo	Reescritura (agregación anticipada por categoria_id + join final) + índice de cobertura	4387	2052	2,14×

Propuestas de la IA que no funcionaron

Propuesta	Por qué se descartó
Índice en pedido(cliente_id, fecha)	No cambió el plan ni el tiempo real (754→751 ms); el cuello era el spill del HashAggregate (Batches 5).
Índice de cobertura "solo" (sin reescritura)	Redujo buffers pero sin reescritura solo bajaba a ~3530 ms; el cambio determinante fue la reescritura.